ADHI COLLEGE OF ENGINEERING AND TECHNOLOGY

Approved by AICTE, New Delhi, Permanent affiliation status by Anna University, Chennai,
Accredited by NAAC, New Delhi, Recognized U/S 12 (B) & 2 (F) of UGC Act 1956
An ISO Certified Institution

Sankarapuram, Near Walajabad, Kanchipuram District - 631605

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

(2024 - 2025)

STUDENT NAME	
REGISTER NO.	
SUBJECT CODE	**CCS366**
SUBJECT NAME	**SOFTWARE TESTING AND AUTOMATION**
YEAR / SEMESTER	**III / VI**

ADHI COLLEGE OF ENGINEERING AND TECHNOLOGY

Approved by AICTE, New Delhi, Permanent affiliation status by Anna University, Chennai,
Accredited by NAAC, New Delhi, Recognized U/S 12 (B) & 2 (F) of UGC Act 1956
An ISO Certified Institution

Sankarapuram, Near Walajabad, Kanchipuram District - 631605

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

LABORATORY RECORD NOTE BOOK
2024 – 2025

REGISTER NO:

This is to certify that this bonafide record of the work done by

Mr./Ms._____ of the year _____ B.E / **B.Tech.,** Department of **ARTIFICIAL INTELLIGENCE AND DATA SCIENCE** in the **CCS366SOFTWARE TESTING AND AUTOMATION**Laboratory in the _____Semester during the year 2024- 2025.

Staff In-Charge **Head of the Department**

Submitted to the University Examination held on _____

Internal Examiner **External Examiner**

Index

Ex No	Date	Name of the Experiment	Page No	Signature
1		Test plan for testing an e-commerce web/mobile application (www.amazon.in).		
2		Test cases for testing the e-commerce application		
3		Test the e-commerce application and report the defects in it.		
4		Test plan and design the test cases for an inventory control system.		
5		Test cases against a client server or desktop application and identify the defects.		
6		The performance of the e-commerce application.		
7		Automate the testing of e-commerce applications using Selenium.		
8		Integrate testing with the above test automation.		
9		Mini Project: a) Build a data-driven framework using Selenium and TestNG b) Build Page Object Model using Selenium and TestNG c) Build BDD framework with Selenium, TestNG and Cucumber		

Exp no: 1	**Develop the test plan for testing an e-commerce web/mobile application (www.amazon.in).**
Date:	

Aim:

To Develop the test plan for testing an e-commerce web/mobile application (www.amazon.in).

Algorithm:

Step 1 : Launch the e-commerce mobile app.

Step 2 : Navigate to the "Sign Up" or "Register" page.

Step 3 : Fill in the required registration details (e.g., name, email, password).

Step 4 : Click the "Register" button.

Step 5 : Verify that the user is redirected to the "Welcome" or "Home" page.

Step 6 : Check if a confirmation email is sent to the registered email address.

Program:

```
import datetime


# Define the Test Plan
class EcommerceTestPlan:
def _init_(self, project_name, project_description):
self.project_name = project_name
self.project_description = project_description
self.creation_date = datetime.date.today()
self.test_cases = []


defadd_test_case(self, test_case):
self.test_cases.append(test_case)
```

```
def generate_report(self):
print(f"Test Plan: {self.project_name}")
print(f"Description: {self.project_description}")
print(f"Creation Date: {self.creation_date}\n")
print ("Test Cases:")
    for idx, test_case in enumerate (self.test_cases, start=1):
print (f"Test Case {idx}:")
        print(test_case)
        print("\n")


# Define a Test Case class
class TestCase:
def _init_(self, title, description, steps, expected_results):
self.title = title
self.description = description
self.steps = steps
self.expected_results = expected_results

def _str_(self):
    return f"Title: {self.title}\n" \
f"Description: {self.description}\n" \
f"Steps:\n{self.format_steps()}\n" \
f"Expected Results:\n{self.format_expected_results()}\n"

defformat_steps(self):
    return "\n".join([f"{i + 1}. {step}" for i, step in enumerate(self.steps)])
```

```
defformat_expected_results(self):
        return "\n".join([f"{i + 1}. {result}" for i, result in enumerate(self.expected_results)])
# Create a sample test plan
if _name_ == "_main_":
test_plan = EcommerceTestPlan(
project_name="Amazon.in Testing",
project_description="Test plan for testing the Amazon.in e-commerce platform."
    )

    # Define and add test cases
    test_case1 = TestCase(
        title="User Registration",
        description="Verify that users can register on the platform.",
        steps=[
            "Navigate to the Amazon.in registration page.",
            "Fill in the registration form with valid user details.",
            "Click the 'Register' button."
        ],
expected_results= [
            "User should be registered successfully.",
            "User should be redirected to the home page.",
        ]
    )

    test_case2 = TestCase(
        title="Search for a Product",
        description="Verify that users can search for products.",
        steps= [
            "Navigate to the Amazon.in home page.",
```

```
            "Enter a product name in the search bar.",
            "Click the 'Search' button."
        ],
expected_results= [
            "Search results for the product should be displayed.",
        ]
    )

    # Add test cases to the test plan
test_plan.add_test_case(test_case1)
test_plan.add_test_case(test_case2)

    # Generate and print the test plan report
test_plan.generate_report()
```

Output:

Test Plan: Amazon.in Testing

Description: Test plan for testing the Amazon.in e-commerce platform.

Creation Date: 2023-09-07

Test Cases:

Test Case 1:

Title: User Registration

Description: Verify that users can register on the platform.

Steps:

1. Navigate to the Amazon.in registration page.
2. Fill in the registration form with valid user details.
3. Click the 'Register' button.

Expected Results:

1. User should be registered successfully.

2. User should be redirected to the home page.

Test Case 2:

Title: Search for a Product

Description: Verify that users can search for products.

Steps:

1. Navigate to the Amazon.in home page.

2. Enter a product name in the search bar.

3. Click the 'Search' button.

Expected Results:

1. Search results for the product should be displayed.

Result:

Thus, the above experiment of developing the test plan for testing an e-commerce web/mobile application has been executed successfully.

Exp no.: 02	Design the test cases for testing the e-commerce application
Date:	

Aim:

To Design the test cases for testing the e-commerce application

Algorithm:

STEP 1 :Verify the Login and Registration.

STEP 2 :Verify the Product Search and Filtering.

STEP 3 :Verify the Product Details and Description.

STEP 4 :Verify the Shopping Cart and Checkout Process.

STEP 5 :Verify the Payment Gateway Integration.

STEP 6 :Verify the Order Tracking and Delivery.

Program:

```
class EcommerceApp:
def _init_(self):
    # Initialize the application and user data
self.users = {}
self.products = {}


def register_user(self, username, email, password):
    # Simulate user registration
    if email not in self.users:
self.users[email] = {'username': username, 'password': password}
        return True
    else:
        return False
```

```
def login_user(self, email, password):
        # Simulate user login
        if email in self.users and self.users[email]['password'] == password:
            return True
        else:
            return False

def search_product(self, product_name):
        # Simulate product search
        if product_name in self.products:
            return self.products[product_name]
        else:
            return None

def add_to_cart(self, user_email, product_name):
        # Simulate adding a product to the cart
        if user_email in self.users and product_name in self.products:
user_cart = self.users[user_email].get('cart', [])
user_cart.append(product_name)
self.users[user_email]['cart'] = user_cart
            return True
        else:
            return False

def checkout(self, user_email, shipping_info, payment_info):
        # Simulate the checkout process
```

```
        if user_email in self.users and 'cart' in self.users[user_email]:
cart_contents = self.users[user_email]['cart']
total_amount = sum(self.products[product_name]['price'] for product_name in cart_contents)


            # Simulate successful payment
            if total_amount == payment_info['amount']:
self.users[user_email]['cart'] = []
                return True
        return False
# Example usage of the EcommerceApp class
if _name_ == "_main_":
    app = EcommerceApp()


    # Test Case 1: User Registration
user_registered = app.register_user("John", "john@example.com", "password123")
    if user_registered:
print("Test Case 1: User Registration - Passed")
    else:
print("Test Case 1: User Registration - Failed")


    # Test Case 2: User Login
user_logged_in = app.login_user("john@example.com", "password123")
    if user_logged_in:
print("Test Case 2: User Login - Passed")
    else:
print("Test Case 2: User Login - Failed")
```

```
    # Test Case 3: Product Search
app.products = {"Laptop": {"price": 1000}, "Phone": {"price": 500}}
    product = app.search_product("Laptop")
    if product:
print("Test Case 3: Product Search - Passed")
    else:
print("Test Case 3: Product Search - Failed")


    # Test Case 4: Add to Cart
app.users = {"john@example.com": {"username": "John", "password": "password123"}}
added_to_cart = app.add_to_cart("john@example.com", "Laptop")
    if added_to_cart:
print("Test Case 4: Add to Cart - Passed")
    else:
print("Test Case 4: Add to Cart - Failed")


    # Test Case 5: Checkout Process
checkout_successful = app.checkout("john@example.com", {"address": "123 Main St."}, {"amount":
1000})
    if checkout_successful:
print("Test Case 5: Checkout Process - Passed")
    else:
print("Test Case 5: Checkout Process - Failed")
```

Output:

Test Case 1: User Registration - Passed

Test Case 2: User Login - Passed

Test Case 3: Product Search - Passed

Test Case 4: Add to Cart - Passed

Test Case 5: Checkout Process – Passed

Result:

Thus, the above program on designing the test cases for testing the e-commerce application has been executed successfully.

Exp no.: 03	**Test the e-commerce application and report the defects in it.**
Date:	

Aim:

To test the e-commerce application and report the defects in it..

Algorithm:

STEP 1 : Start the program.

STEP 2 : Import the required selenium libraries and any other testing frameworks or libraries you might need.

STEP 3 : Choose the webdriver(e.g Chrome, Firefox) and initialize it.

STEP 4 : Use the Webdriver to open the e-commerce websites URL.

STEP 5 : Find an internet with web applications l(e.g,online forms, shopping carts, video streaming, social media, games, and e-mail.)

STEP 6 : Close the webdriver instance when testing is completed.

STEP 7 :Create separate test cases for different scenarios (e.g, login, product search, checkboard) and repeat steps 3 and 5.

STEP 8 : Execute your test cases using a testing to find defects in web applications.

STEP 9 : Stop the program.

Program:

```
class EcommerceApp:
def _init_(self):
    # Initialize the application and user data
self.users = {}


def register_user(self, username, email, password):
    # Simulate user registration
    if email not in self.users:
self.users[email] = {'username': username, 'password': password}
        return True
    else:
```

```
            return False

    def login_user(self, email, password):
        # Simulate user login
        if email in self.users and self.users[email]['password'] == password:
            return True
        else:
            return False



def test_login():
    app = EcommerceApp()

    # Test Case 1: Valid User Login
app.register_user("John", "john@example.com", "password123")
    result = app.login_user("john@example.com", "password123")
    if result:
print("Test Case 1: Valid User Login - Passed")
    else:
print("Test Case 1: Valid User Login - Failed")

    # Test Case 2: Invalid User Login
    result = app.login_user("invalid@example.com", "invalidpassword")
    if not result:
print("Test Case 2: Invalid User Login - Passed")
    else:
print("Test Case 2: Invalid User Login - Failed")
if _name_ == "_main_":
print("Running E-commerce Application Tests")
test_login()
```

Output:

Running E-commerce Application Tests

Test Case 1: Valid User Login - Passed

Test Case 2: Invalid User Login – Passed

Result:

Thus, the above programs to test the e-commerce application and report the defects in it has been executed successfully.

Exp no.: 04	**Develop the test plan and design the test cases for an inventory control system.**
Date:	

Aim:

To develop the test plan and design the test cases for an inventory control system.

Algorithm:

STEP 1 : Define Test Objectives: Determine the primary objectives of testing.

STEP 2 : Identify Test Scenarios: List various scenarios to be tested

STEP 3 : Select Test Data: Prepare sample data sets that cover different scenarios.

STEP 4 : Design Test Cases: Create detailed test cases for each identified scenario, specifying preconditions, steps, and expected outcomes.

STEP 5: Execute Test Cases: Execute the test cases using the prepared data .

STEP 6:Analyze Results: Evaluate test results to identify defects, issues, and areas for improvement.

STEP 7: Report and Retest: Document and report any identified defects, and then retest once the issues are resolved.

Test Plan for Inventory Control System

1. Introduction

Project Name: Inventory Control System Testing

Objective: To ensure the functionality, reliability, and accuracy of the inventory control system.

2. Test Scope

Identify the modules, features, and functionalities to be tested.

Define what is out of scope for this testing phase.

3. Test Objectives

List the specific goals and objectives of the testing process.

4. Test Environment

Describe the hardware and software environment for testing.

5. Test Strategy

Explain the overall approach to testing, including manual vs. automated testing, regression testing, and exploratory testing.

6. Test Schedule

Define the testing timeline, milestones, and deadlines.

7. Test Deliverables

List the documents and artifacts to be produced during testing (e.g., test cases, test reports).

8. Test Cases

Define test cases to be executed during the testing phase.

9. Defect Management

Describe how defects will be reported, tracked, and managed.

10. Risks and Contingencies

Identify potential risks to the testing process and provide contingency plans.

Sample Test Cases

Test Case 1: Adding Items to Inventory

Objective:

To verify that items can be successfully added to the inventory.

- Log in to the inventory control system.
- Click on the "Add Item" button.
- Enter valid item details (e.g., name, quantity, category).
- Click the "Save" button.

Expected Result:

The item should be added to the inventory, and the quantity should be updated accordingly.

Test Case 2: Updating Item Information

Objective:

To verify that item information can be updated correctly.

- Log in to the inventory control system.
- Search for an existing item.
- Click on the item to edit its details.
- Modify item information (e.g., update quantity, change category).
- Click the "Save" button.

Expected Result:

The item's information should be updated in the inventory.

Test Case 3: Inventory Reporting

Objective:

To verify the accuracy of inventory reporting.

- Log in to the inventory control system.
- Access the "Inventory Report" feature.
- Check that the reported item quantities match the actual quantities in the inventory.

Expected Result:

The inventory report should accurately reflect the current state of the inventory.

Test Case 4: Stock Alerts

Objective:

To ensure that stock alerts are generated when inventory falls below a certain threshold.

- Log in to the inventory control system.
- Check an item with a low quantity.
- Verify that a stock alert is displayed.

Expected Result:

A stock alert should be generated for items with quantities below the threshold.

Test Case 5: User Permissions

Objective:

To verify that user permissions are enforced correctly.

- Log in with different user roles (e.g., admin, regular user).
- Attempt to perform actions that require admin permissions (e.g., adding or deleting items).
- Verify that admin users can perform these actions, while regular users cannot.

Expected Result:

User permissions should be enforced as specified.

Result:

Thus, the above program to develop the test plan and design the test cases for an inventory control system was implemented and executed successfully.

Exp no.: 05	**Execute the test cases against a client server or desktop application and identify the defects**
Date:	

Aim:

To execute the test cases against a client server or desktop application and identify the defects.

Algorithm:

STEP 1: Test Case Preparation:Start by gathering all the test cases you need to execute.

STEP 2: Test Environment Setup:Ensure that you have a suitable test environment set up.

STEP 3:Test Data Preparation:Prepare test data that will be used during the test.

STEP 4:TestExecution:Execute the test cases one by one.

STEP 5: Defect Identification:While executing the test cases, if you encounter any unexpected behaviour.

STEP 6:DefectReporting:For each identified defect, create a defect report.

Program:

```
import requests


# Define the base URL of the server or application under test
BASE_URL = "http://localhost:8080"  # Replace with the actual server URL


# Function to send an API request to the server
defsend_request(endpoint, method="GET", data=None):
url = f"{BASE_URL}/{endpoint}"
   if method == "GET":
       response = requests.get(url)
elif method == "POST":
       response = requests.post(url, json=data)
```

```
    # Add more HTTP methods as needed (e.g., PUT, DELETE)
    return response
# Sample Test Cases
def test_case_1():
    # Test Case: Verify successful user registration
    endpoint = "register"
    data = {"username": "testuser", "password": "testpassword"}
    response = send_request(endpoint, method="POST", data=data)

    if response.status_code == 200:
print("Test Case 1: User Registration - Passed")
    else:
print("Test Case 1: User Registration - Failed")
def test_case_2():
    # Test Case: Verify user login
    endpoint = "login"
    data = {"username": "testuser", "password": "testpassword"}
    response = send_request(endpoint, method="POST", data=data)
    if response.status_code == 200:
print("Test Case 2: User Login - Passed")
    else:
print("Test Case 2: User Login - Failed")

def test_case_3():
    # Test Case: Verify product retrieval
    endpoint = "products"
    response = send_request(endpoint)
    if response.status_code == 200:
print("Test Case 3: Product Retrieval - Passed")
    else:
print("Test Case 3: Product Retrieval - Failed")
```

```
# Main test execution
if _name_ == "_main_":
print("Executing Test Cases for Client-Server Application")
    # Execute test cases
    test_case_1()
    test_case_2()
    test_case_3()
    # Additional test cases can be added here
print("Test Execution Completed")
```

Output:

```
Executing Test Cases for Client-Server Application
Test Case 1: User Registration - Failed
Test Case 2: User Login - Failed
Test Case 3: Product Retrieval - Failed
Test Execution Completed
```

Result:

Thus, the program to execute the test cases against a client server or desktop application and identify the defects.

Exp no.: 06 Date:	Test the performance of the e-commerce application.

Aim:

To Test the performance of the e-commerce application.

Algorithm:

STEP 1: Open the e-commerce mobile app on the testing device.

STEP 2: Tap on the "Sign Up" or "Register" option in the app's navigation menu.

STEP 3: Enter a valid name, a unique email address, and a secure password in their respective fields.

STEP 4: Tap the "Register" button to submit the registration form.

STEP 5: Confirm that the app navigates to the "Welcome" or "Home" page, indicating a successful registration.

STEP 6: Access the registered email inbox.

Verify the presence of a confirmation email.

Confirm that the email contains the necessary registration confirmation details.

Program:

```
from locust import HttpUser, task, between

class EcommerceUser(HttpUser):

wait_time = between(1, 3)  # Random wait time between requests

    @task

defview_product(self):

product_id = 1  # Replace with a valid product ID

self.client.get(f"/product/{product_id}")

    @task

defsearch_product(self):

search_term = "laptop"  # Replace with a valid search term
```

```
self.client.get(f"/search?query={search_term}")
    @task
defadd_to_cart(self):
product_id = 1  # Replace with a valid product ID
self.client.post(f"/add-to-cart/{product_id}", json={"quantity": 1})
    @task
def checkout(self):
self.client.get("/checkout")
if _name_ == "_main_":
    import os
os.system("locust -f performance_test.py")
```

Result:

Thus, the program to Test the performance of the e-commerce application has been executed successfully.

Exp no.: 07 Date:	**Automate the testing of e-commerce applications using Selenium**

Aim:

To Automate the testing of e-commerce applications using Selenium.

Algorithm:

STEP 1: Start the program.

STEP 2 :Import the required selenium libraries and any other testing frameworks or libraries needed.

STEP 3: Choose the webdriver(e.g Chrome, Firefox) and initialize it.

STEP4 : Use the Webdriver to open the e-commerce websites URL.

STEP5 :Find and internet with web elements (e.g, clicking- buttons, filling forms)

STEP6 :Close the webdriver instance when testing is completed.

STEP7 : Create separate test cases for different scenarios (e.g, login, product search, checkboard) and repeat steps 3 and 5.

STEP8 : Execute your test cases using a testing framework or test runner.

STEP9 :Stop the program.

Program:

```
from locust import HttpUser, task, between


    class EcommerceUser(HttpUser):
    wait_time = between(1, 3)  # Random wait time between requests


       @task
    defview_product(self):
    product_id = 1  # Replace with a valid product ID
    self.client.get(f"/product/{product_id}")
```

```
    @task
    defsearch_product(self):
    search_term = "laptop"  # Replace with a valid search term
    self.client.get(f"/search?query={search_term}")

    @task
    defadd_to_cart(self):
    product_id = 1  # Replace with a valid product ID
    self.client.post(f"/add-to-cart/{product_id}", json={"quantity": 1})

    @task
    def checkout(self):
    self.client.get("/checkout")

    if _name_ == "_main_":
    import os
os.system("locust -f performance_test.py")
```

Result:

Thus the program to Automate the testing of e-commerce applications using Selenium has been executed successfully.

Exp no: 08	**Integrate TestNG with the above test automation**.
Date:	

Aim:

To Integrate TestNG with the above test automation.

Algorithm:

STEP 1: Start the program.
STEP 2 :Create a testNG test class.
STEP 3: Add dependencies.
STEP 4:Configure testNG XML file.
STEP 5: Annotate test methods in TestNG.
STEP 6:Run the test to execute your test.
STEP 7:View test results.
STEP 8:Stop the program.

Program:

```
from selenium import webdriver
from selenium.webdriver.common.keys import Keys

# Create a WebDriver instance (for Chrome)
driver = webdriver.Chrome(executable_path="path/to/chromedriver")

# Define the e-commerce website URL
website_url = "https://www.example.com” # Replace with the actual URL

# Function to perform e-commerce actions
deftest_ecommerce():
    try:
        # Open the e-commerce website
driver.get(website_url)

        # Perform a search for a product
search_box = driver.find_element_by_name("q")  # Replace with the actual search element locator
search_box.send_keys("product name")  # Replace with the product name
search_box.send_keys(Keys.RETURN)

        # Click on a product from the search results
product_link = driver.find_element_by_css_selector(".product-link")  # Replace with the actual
CSS selector
product_link.click()
```

```
        # Add the product to the cart
add_to_cart_button = driver.find_element_by_id("add-to-cart")  # Replace with the actual button locator
add_to_cart_button.click()

        # Verify that the product is in the cart
cart_icon = driver.find_element_by_id("cart-icon")  # Replace with the actual cart icon locator
cart_icon.click()

        # Assert that the product is in the cart
        assert "Product Name" in driver.page_source  # Replace with the product name

print ("Test Passed: Product added to the cart successfully!")

    except Exception as e:
print ("Test Failed:", str(e))

    finally:
        # Close the browser
driver.quit()

# Run the e-commerce test
test_ecommerce()
```

Result:

Thus, the program to Integrate TestNG with the above test automation has been executed successfully.

Ex no: 09	Mini Project
Date:	

Aim:

a) Build a data-driven framework using Selenium and TestNG
b) Build Page Object Model using Selenium and TestNG
c) Build BDD framework with Selenium, TestNG and Cucumber

Algorithm:

STEP 1: Start the program.
STEP 2:Create a testNG test class.
STEP 3: Add dependencies.
STEP 4:Build a data-driven framework using Selenium
STEP 5: Annotate test methods in TestNG.
STEP 6: Annotate test methods in Cucumber.
STEP 7:Run the test to execute your test.
STEP 8:View test results.
STEP 9:Stop the program.

Program:

a) Build a data-driven framework using Selenium and TestNG

Step 1:

- Create a new python project in your preffered IDE.
- Install the required libraries:selenium, testing and pytest
- Create a config folder for storing configuration files.
- Create a data folder for storing test data.
- Create a page's folder for storing page objects.

Step 2:

- Create a config. json file in the config folder to storecommunication data.
- Create a test – data .csv file in the data folder to store the test file data.
- Create a base – test .py file in the test folder to define test class for the login page.

Config.json:

```
{
        Url:https:// example.com
        "Username": "username":
        "Password":"Password"
  }
```

Base_test .py

```
    import pytest
    from selenium impact webdriver from selenium.web
    import by from selenium.webdriver . support
    import expected _ conditions as EC

Class base test:
        Def setup _method(self):
Self.driver = webdriver.chrome()
Self.driver.get ("https://example.com"):
        Def tcardown_method(self):
Self.driver.quit )
```

Test_login.py

```
    import_pytest from base_test
    import base test from pages.login_page
    import login page

   Class test login (base test):
        Def test_login(self):
Login_page = login page (self.driver)
Login_page.enter_username (" testuser ")
Login_page.enter_password (" testpassword ")
```

b) Build Page Object Model using Selenium and TestNG

Step 1 : Create page objects .

- ❖ Create a login_page.py file in the pages folder to define a page object for the login page.
- ❖ Create a home_page.py file in the pages of folder to define a page object for the home page.login.py

```
from selenium.webdriver.common.by
        import by from selenium.webdriver.support.vi
        import webdriver wait from selenium.webdriver.support
        import expected_conditons as EC

         Class login page:
                Def_int_(self,driver) :
                Self.driver = driver self.username_input = ( by.NAME , 'username')
                Self.password_input = (by.NAME = 'password')
          Def enter_username / self,username ):
                        Self.driver.find element(*self.username_input) send_keys(username)
```

```
        Def enter_password(self,password):
            Self.driver.find element(*self.username_input) send_keys(username)
    Def click_login_button(self):
            Self.driver.find element(*self.login_button).Click()


Class home page:
        Def_int_(self,driver) :
            Self.driver = driver
            Self.logout_button=(by.name," logout")
        Def click_logout_button(self):
            Self.driver.find element(*self.login_button).Click()
```

c) Build BDD framework with Selenium, TestNG and Cucumber

Step 1: Install the cucumber library using pip: pip install cucumber.
Step 2: Create a login features file in the features folder to define a feature for the login page**.**

Login features

Feature: login
As a user
I want to login to the application so that I can access the home page

Scenario:
Successful login
Given I am on the login page when I enter valid username and password.

Scenario:
Unsuccessful login
Given I am on the login page when I enter invalid username and password.

Step3:

- ❖ Create a login steps py file in the steps folder to define step definitions for the login features.

Login steps.py

From behave import given when then from pages' login page import login page

```
@given ("I am on the login page:")
        Def step-impl(context):
        Context.login_page=login page (context. Driver)
@when ("enter valid username and password")
        Def step-impl(context):
        context
```

Result:

Thus, the program to a data-driven frameworkPage Object Model and BDD framework TestNG (cucumber) with the above test automation has been executed successfully.